4 2

1 1 4 1 3 4

PHASE _REVIEW_R — Independent § . . re-review of the `anti_bluff` remediation

Field	Value
Revision	1
Created	2026-07-23
Last modified	2026-07-23T05:10:00Z
Reviewer	(T1/main - claude1) — independent §11.4.142/§11.4.194 re-review, §11.4.209 substrate: Fable (claude-fable-5) at xhigh effort
Scope	`constitution/submodules/anti_bluff/` @ HEAD `e841271a080de822d40cf58b758acaddb4374788` (two forward-only commits `35167e8` fixes + `e841271` evidence on top of the reviewed-NO-GO `1e193e1`)
Prior round	`PHASE4_REVIEW.md` — NO-GO (2 IMPORTANT + 4 NIT)
Method	TRUST-NOTHING: every remediation claim independently re-executed on this host (real temp SQLite DBs, no mocks §11.4.27; control-needled scans §11.4.201(7)(b); own `ls-remote`)
Substrate facts	bash 5.2.37, sqlite3 3.50.6, host `/mnt/track1/atmosphere-t1`

VERDICT: GO — Blocking / Important / Nit (none gating)

I independently re-ran the INSERT RED→GREEN and the full 5-suite / 52-check battery myself and found **zero functional, zero bluff-capable, zero seam-weakening findings**. All six prior-round findings are closed (NIT-3 by confirmed absence from this tree — it lives in constitution-layer docs, out of scope for this slice per the conductor's carve-out). Two NEW nit-level doc-comment findings were identified (N1, N2 below); both are prose-only, both paths are proven fail-closed at runtime, neither weakens any seam — following this project's own recorded review convention ("GO: 0 Blocking / 0 Important / 3 Minor, none gating", constitution Rev-63 status summary), they are fix-forward material, not loop re-arming. If the conductor elects the strictest §11.4.134 zero-warnings reading, both are closable in one trivial doc-only forward commit before the gitlink bump; no code change is required.

Per §11.4.134 this GO is backed by captured physical evidence: every claim below was re-executed in this session; artifacts under `/tmp/ab_review_r2/` (throwaway; the load-bearing transcripts are reproduced inline).

. IMPORTANT- — INSERT custody bypass: CLOSED (independently re-proven)

. RED independently re-derived on the pre-fix artifact (not read from the report)

`git archive 1e193e1` → fresh tree → `custody_schema.sql` + its own `apply_custody.sh --no-probe` (real temp SQLite DB):

- Pre-fix template carries **5** triggers, **0** `BEFORE INSERT` triggers.
- `INSERT INTO items(...,status='Fixed')` with NO chain → **ACCEPTED, exit 0, 1 row landed, 0 history rows** — the PC-1 bypass live through the INSERT door.
- `INSERT ... status='Reopened'` unstaged → **ACCEPTED**.

Matches `test/evidence/INSERT_RED.txt` (A1/A2/B2/E1/E2/F1 FAIL) exactly.

. GREEN independently re-run on `e841271`

Fresh DB from HEAD artifacts, `apply_custody.sh` verifies **8/8** triggers. Suite `test_insert_custody.sh` green inside the full run (§6). My own raw-SQL probes (§1.3) confirm the same on a DB I built myself.

. Adversarial § . . every-factor battery (all independently executed, real DB)

Probe	Path	Result
P1	plain `INSERT` at terminal, no chain	**REFUSED** (`— INSERT path` message), 0 rows
P2	ordinary open-item `INSERT`	**ACCEPTED untouched**, 0 history rows (§11.4.201(1) guard holds)
P3	full-custody-chain `INSERT`-at-terminal	**ACCEPTED** + exactly 1 auto-audit row
P4	`INSERT OR REPLACE` (new row, terminal)	**REFUSED**, 0 rows
P5	`REPLACE INTO` existing open row → terminal	**REFUSED**, row stays `In progress`
P6	`INSERT OR IGNORE` (terminal)	**REFUSED** (RAISE(ABORT) overrides OR IGNORE on sqlite 3.50.6), 0 rows
P7	UPSERT, terminal insert-arm	**REFUSED** (INSERT twin), 0 rows
P8	UPSERT, open insert-arm + `DO UPDATE SET status='Fixed'`	**REFUSED** (the DO-UPDATE arm fires the **UPDATE** trigger — message without `— INSERT path`), row unchanged
P9	multi-row `INSERT` (one open + one un-evidenced terminal)	**REFUSED**, **0 partial rows** (ABORT rolls back the whole statement)
P10	`INSERT INTO items SELECT ...` from staging table	**REFUSED**, 0 rows

Probe	Path	Result
P11	**item_history smuggle**: forge a `'Fixed'` event row for the SAME id first, then terminal `INSERT`	history append ACCEPTED (the ledger's one legit write — by design), terminal INSERT **STILL REFUSED** — the chain is checked in `guard_registry` + `verdicts`, never in `item_history`. The no-INSERT-twin decision for `item_history` is **SOUND**: it cannot be leveraged into a terminal `items` row.
P12	`UPDATE OR REPLACE` → terminal / plain `UPDATE` → terminal	both **REFUSED** — UPDATE seam did not regress (also case D1 in the suite)
P13	born-`Reopened` unstaged / staged	unstaged **REFUSED**; staged **ACCEPTED** with `by_actor=User` recorded + intake `consumed=1`
P14	`item_history` UPDATE / DELETE	both **REFUSED** (append-only guards intact)

Zero partial rows in every refused case (final census: only `AB-LEGIT|Fixed`, `AB-OPEN|In progress`, `AB-R2|Reopened`). Fix-A-creates-B (§11.4.1): none found.

. Raw-SQL write-path enumeration vs the README § / install.sh claim — HONEST AND EXACT (§ . .)

Covered (claimed AND proven): `items.status` via every `INSERT` variant (plain, `OR REPLACE`, `OR IGNORE`, UPSERT insert-arm, multi-row, `INSERT..SELECT`) and every `UPDATE` variant (plain, `OR REPLACE`, UPSERT DO-UPDATE arm); `item_history` `UPDATE` + `DELETE`. Residuals (claimed NOT covered — each one **proven REAL by me**, so the claim is neither overbroad nor under-stated):

- `DELETE FROM items` on a terminal item → ACCEPTED, orphans its history (disclosed §4 + tracked OWED README §5 item 7).
- Fabricated chain rows (`guard_registry` / `verdicts` direct INSERT) → enables a terminal insert (disclosed §4 first bullet — shape-not-provenance by design; provenance is §11.4.115(F)/SOL-02 territory, honestly layered).

- `DROP TRIGGER` → ACCEPTED for a raw writer (disclosed); sweep `--require-triggers` C0 then reports `7/8 custody triggers` and FAILs — verified live.

No undisclosed write path found (ATTACH does not bypass same-DB triggers; byte-level/ `writable_schema` editing falls under the disclosed determined-writer/ DROP-TRIGGER class).

. IMPORTANT- — coverage floor: CLOSED (instrument real, figure honest)

- `test/coverage_report.sh` is a real instrument, not a stub: re-run by me on the committed `e841271` tree → **TOTAL 446/830 = 53%**, per-file figures IDENTICAL to the committed `test/evidence/COVERAGE_20260723.txt` (42/70/71/61/47/51/73/40/55/52/49/53%), suite exit 0 under trace.
- **Control needle verified both directions** (§11.4.201(7)(b)): (a) healthy path — my run emitted a report only after the internal needle sighted COV lines for `run_all.sh`; (b) golden-bad — in a mutated THROWAWAY copy (fd 9 routed to /dev/null; the repo untouched) the instrument returned `INSTRUMENT-BLIND: trace carries no COV lines for run_all.sh` **exit 2 and refused to emit any figure**. The clean-tree precondition also fires (separately observed, exit 2 on a dirty tree).
- Honest limits stated in-instrument and in README §5 item 6: line-not-branch, `set +x` /traps unaccounted, block terminators + continuation lines + heredoc bodies undercount → conservative LOWER BOUND; kcov/bashcov/bats probed ABSENT; 0 Go sources = honest §11.4.3 skip.
- **No fabricated ≥85% anywhere** (control-needled scan; README explicitly: "the recorded 53% is NOT a claim the true figure meets or misses the 85% floor").
- §11.4.224(E) fence: checked-in `test/coverage_exclusions.txt`, ONE entry (the instrument itself, first-party, justification + §11.4.197 OWED item in README §5 item 6), printed as an honest gap on every run — observed live.
- OWED items genuinely filed: README §5 items 6 (calibration + branch-capable instrument + self-measurement) and 7 (row-DELETE custody).

- Note (not a finding): the committed report's tail carries the `35167e8`-era 2-line limits wording while the instrument at `e841271` prints the extended 4-line wording — the report honestly stamps its generating commit (`@ 35167e8`) and my `e841271` re-run reproduces the figures exactly.

. NIT- — project literals: CLOSED

Control-needled tree scan (alternation ERE, `--exclude-dir=.git`): `atmosphere|rk3588|rockchip|kinopoisk|orangepi|arvus|presenter|VideoOutputManager|RouteToSecondary|9261537|998fd36615e99484|66ff9c4f51f00ee7|/mnt/track|helixqa|smarttube|mistiq|vader` → **0 hits**; same-class needle (`CUSTODY-REFUSED|<absent-token>`) sighted through the same instrument+path → the zero is certified. `VideoOutputManagerService` / `RouteToSecondary` / `9261537` are gone (fixtures now `ExamplePlayerService.java` / `build-2026-07-23-abcdef`).

Disclosed remainder judged: the `atm_id` column + `ATM-` / `AB-` test ids are **constitution-vocabulary, not a project leak** — §11.4.54's own universal anchor defines "ATM-NNN ticket identifier" at the constitution layer, and `custody_schema.sql` line 2-3 declares table/column names consumer-mappable DATA per §11.4.35. The disclosure is honest.

. NIT- — `nq_stream_contains` needle: CLOSED (RED re-derived, GREEN verified)

- Contract read + verified in code: `0 PRESENT | 1 CERTIFIED-ABSENT | 2 INSTRUMENT-BLIND | 3 NEEDLE-CLASS-MISMATCH` ; CERTIFIED-ABSENT is emitted ONLY after the class-checked needle is grepped through the SAME captured stream file (`lib/needle.sh:116-125`).
- **RED independently re-derived**: HEAD's `test_needle.sh` run against the PRE-FIX (`1e193e1`) `lib/needle.sh` → H1/H3/H4 FAIL with the needle-less `ABSENT:` output, byte-matching the `INSERT_RED.txt` transcript; exit 1.

- GREEN on HEAD: H1 CERTIFIED-ABSENT(1) with needle sighted, H3 INSTRUMENT-BLIND(2) on an unsighted needle (golden-bad fires), H4 NEEDLE-CLASS-MISMATCH(3) on a class-mismatched needle (golden-bad fires) — inside my own suite runs.

. NIT- — remote fan-out: CLOSED (my own ls-remote)

- `git remote -v` : `origin` fetch = GitHub; push URLs = **BOTH** `git@github.com:vasic-digital/anti_bluff.git` AND `git@gitlab.com:vasic-digital/anti_bluff.git` .
- `git ls-remote` (run by me, both rc=0): GitHub `refs/heads/main` = `e841271a08...` , GitLab `refs/heads/main` = `e841271a08...` — both mirrors at the remediated HEAD; ff by construction (see §6 ancestry).

. Whole-slice adversarial pass: CLEAN

- **Full suite re-run by me, twice**: 5/5 suites, **52 PASS / 0 FAIL**, exit 0 both iterations, outputs **byte-identical** (`diff` empty — §11.4.50). Count matches the claim (needle 11 + status_custody 18 + insert_custody 9 + evidence_class 6 + out_of_box 8 = 52).
- **Mutation markers (§11.4.84)**: control-needled scan for `MUTATED for paired|// always pass|# always pass|_mutated_|# MUTATION|// MUTATION` → 0 hits, needle sighted. Submodule working tree clean (`git status --porcelain` empty).
- **Forward-only**: `git merge-base --is-ancestor 1e193e1 e841271` → TRUE; exactly two commits (`35167e8` , `e841271`) atop the reviewed `1e193e1` ; no history rewrite; both mirrors at `e841271` (§5) — ff per §11.4.113.
- **Parse checks (§11.4.67)**: every tracked `*.sh` clean under `bash -n` AND `sh -n` .
- **Pointer boundary (§11.4.28)**: the constitution repo's gitlink for `submodules/anti_bluff` is still `1e193e1` with the `M` dirty-pointer state — the bump is correctly left to the pointer-owner (conductor); the remediation is submodule-only as claimed.

- **NIT-3 confirmation**: `00:00:00Z` is genuinely ABSENT from this submodule tree (0 hits; same-class needle `23:53:26Z` sighted) — it lives only in the two constitution-layer record docs, out of scope for this slice per the conductor's carve-out. Not blocked on.

. NEW findings (both Nit, both non-gating, fix-forward)

N1 (Nit) — stale trigger count in `custody_sweep.sh` usage header. `seams/status_custody/custody_sweep.sh:9-10` still reads "additionally FAIL when the **5** custody triggers are absent" while the C0 check (and its inline comment) correctly enforce **8**. Introduced by omission in `35167e8` (the header was correct at `1e193e1`). Prose-only; the code is correct (C0 verified live: post-DROP it reports `7/8` and FAILs). §11.4.18/§11.4.6-class doc drift; one-line fix-forward.

**N2 (Nit) — `apply_custody.sh` post-substitution comment overstates the check

- undocumented `&`-vocabulary limitation (fail-closed).** `apply_custody.sh:100-101` comment says the needle verifies the marker is gone AND "the first terminal literal MUST be present"; the code checks only marker-absence. Adjacent measured fact: on bash ≥5.2 (`patsub_replacement` default; this host 5.2.37 confirmed `&` → match expansion), a consumer `AB_TERMINAL_STATUSES` containing `&` (e.g. `Fixed & Verified`) re-injects the marker and the apply **fails CLOSED** — proven live: `AB-FAILED: template substitution left the marker in place`, exit 1, zero partial apply, no custody bypass — but the message does not name the `&` cause. The §11.4.33 closed vocabulary carries no `&`, so this is edge-theoretical; the marker-absence needle is exactly what converts the footgun into an honest refusal. Fix-forward: align the comment with the code (or add the literal-present check) + one line documenting the `&` limitation.

Neither nit weakens a seam, produces a false verdict, or opens a bypass; both degraded paths are proven fail-closed. Per the project's recorded convention (GO with non-gating minors, fix-forward per §11.4.113) they do not re-arm the loop; the conductor MAY fold both into the next doc-only forward commit.

. Anti-bluff certification

- I independently re-ran the INSERT **RED** on the pre-fix `1e193e1` artifacts (raw terminal INSERT ACCEPTED, 1 row, 0 audit rows) and the **GREEN** on `e841271` (every INSERT variant REFUSED, zero partial rows, legit paths untouched) on real temp SQLite DBs — no mocks (§11.4.27).
- I independently re-ran the **full suite twice** (5 suites / 52 checks, byte-identical, exit 0) and the **coverage instrument** (figures reproduce exactly; its control needle proven to fire on a blinded trace).
- Every absence claim in this review is control-needled (§11.4.201(7)(b)) — no zero was read as evidence without a same-class needle sighted through the same instrument and path.
- Both mirror tips verified with my own `ls-remote`, not the report's claim.
- **Zero new Blocking or Important findings. GO.**